Practical 1

Part A	
1)	Vending machine (VendingMachine.java, main in this file) (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN. (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far. (d) Stop the loop once the total reaches 15 or more. (e) Print the change to return (total − 15).
	VendingMachine.java

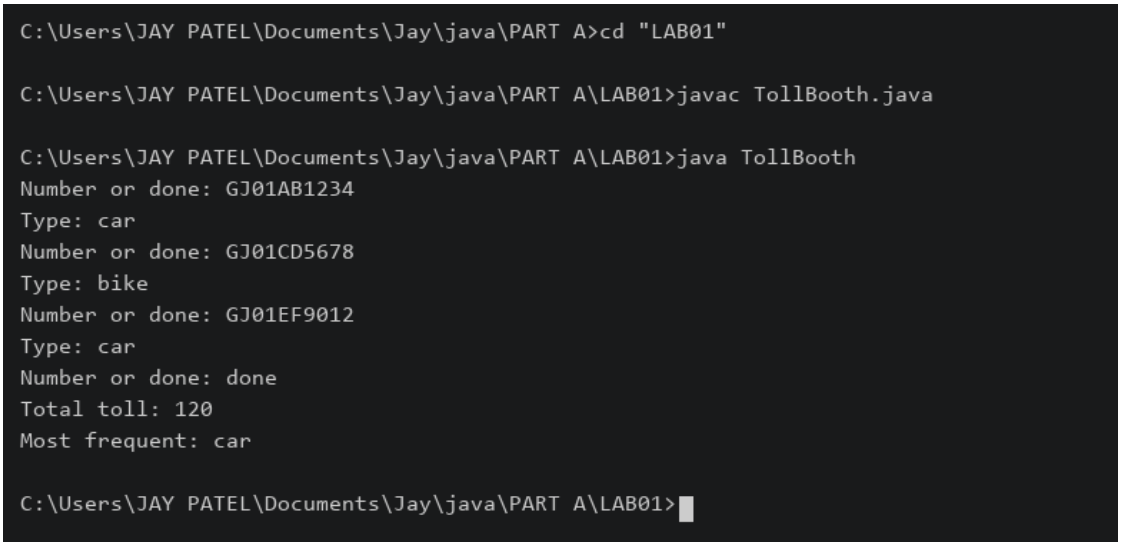
Code	`import java.util.Scanner;`
	`enum Coin { ONE, TWO, FIVE, TEN }`
	`public class VendingMachine {`
	`  public static void main(String[] args) {`
	`  Scanner sc = new Scanner(System.in);`
	`  int price = 15;`
	`  int total = 0;`
	`  while (total < price) {`
	`  System.out.print("Enter coin: ");`
	`  Coin coin = Coin.valueOf(sc.next().toUpperCase());`
	`  int value = switch (coin) {`
	`  case ONE -> 1;`
	`  case TWO -> 2;`
	`  case FIVE -> 5;`
	`  case TEN -> 10;`
	`  };`
	`  total += value;`
	`  System.out.println("Total: " + total);`
	`  }`
	`  System.out.println("Paid. Change: " + (total - price));`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB01"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>javac VendingMachine.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>java VendingMachine`
	`Enter coin: FIVE`
	`Total: 5`
	`Enter coin: TEN`
	`Total: 15`
	`Paid. Change: 0`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>`

2)	Toll booth (TollBooth.java, main in this file) (a) Define a record Vehicle(String number, String type). (b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types “done” for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type’s counter. (d) After the loop, print the total toll and the type with the highest count.

	TollBooth.java

Code	import java.util.Scanner;
	record Vehicle(String number, String type) {}
	public class TollBooth {
	  public static void main(String[] args) {
	  Scanner sc = new Scanner(System.in);
	  int total = 0, bikes = 0, cars = 0, trucks = 0;
	  while (true) {
	  System.out.print("Number or done: ");
	  String number = sc.next();
	  if (number.equalsIgnoreCase("done")) { break; }
	  System.out.print("Type: ");
	  Vehicle v = new Vehicle(number, sc.next().toLowerCase());
	  int toll = switch (v.type()) {
	  case "bike" -> 20;
	  case "car" -> 50;
	  case "truck" -> 150;
	  default -> 0;
	  };
	  switch (v.type()) {
	  case "bike" -> bikes++;
	  case "car" -> cars++;
	  case "truck" -> trucks++;
	  default -> System.out.println("Invalid type");
	  }
	  total += toll;
	  }
	  System.out.println("Total toll: " + total);
	  if (bikes + cars + trucks == 0) {
	  System.out.println("Most frequent: none");
	  } else {
	  int max = Math.max(bikes, Math.max(cars, trucks));
	  if (bikes == max) { System.out.println("Most frequent: bike"); }
	  if (cars == max) { System.out.println("Most frequent: car"); }
	  if (trucks == max) { System.out.println("Most frequent: truck"); }
	  }
	  }
	}

Output	

3)	(For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java) (a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK. (b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases). (c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score. (d) After 5 rounds, print the overall winner.
	RPSLS.java
Code	

```
import java.util.Random;
import java.util.Scanner;
enum Move { ROCK, PAPER, SCISSORS, LIZARD, SPOCK }
public class RPSLS {
   static int winner(Move a, Move b) {
      if (a == b) { return 0; }
      boolean win = switch (a) {
         case ROCK -> b == Move.SCISSORS || b == Move.LIZARD;
         case PAPER -> b == Move.ROCK || b == Move.SPOCK;
         case SCISSORS -> b == Move.PAPER || b == Move.LIZARD;
         case LIZARD -> b == Move.PAPER || b == Move.SPOCK;
         case SPOCK -> b == Move.ROCK || b == Move.SCISSORS;
      };
      if (win) { return 1; }
      return -1;
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      Random random = new Random();
      int player = 0, computer = 0;
      for (int i = 1; i <= 5; i++) {
         System.out.print("Your move: ");
         Move a = Move.valueOf(sc.next().toUpperCase());
         Move b = Move.values()[random.nextInt(5)];
         System.out.println("You: " + a + " Computer: " + b);
         int result = winner(a, b);
         if (result == 1) { player++; System.out.println("You win round"); }
         else if (result == -1) { computer++; System.out.println("Computer wins round"); }
         else { System.out.println("Tie"); }
      }
      if (player > computer) { System.out.print("You win "); }
      else if (computer > player) { System.out.print("Computer wins "); }
      else { System.out.print("Draw "); }
      System.out.println(player + "-" + computer);
   }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB01"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>javac RPSLS.java

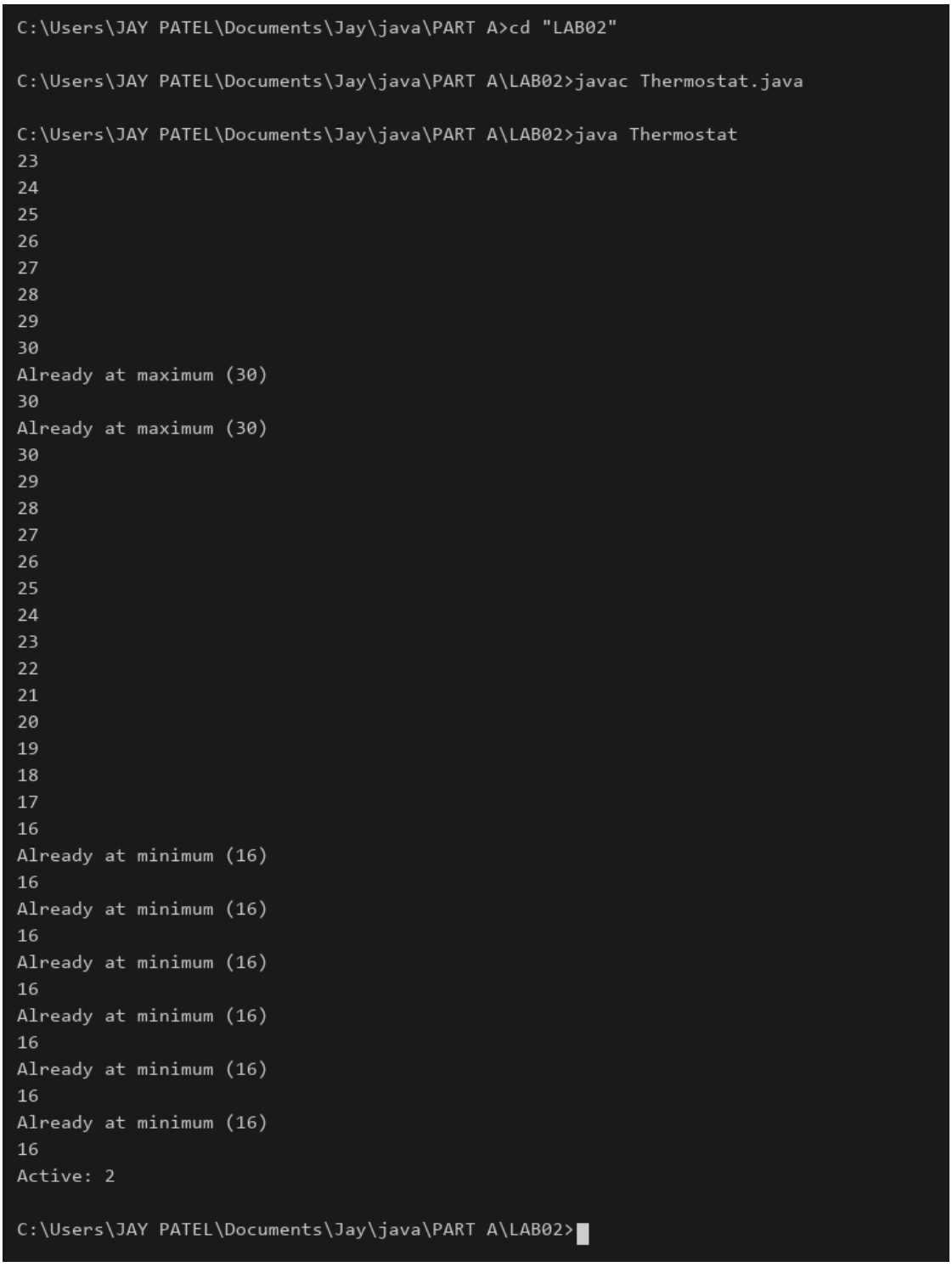
C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>java RPSLS
Your move: ROCK
You: ROCK Computer: SPOCK
Computer wins round
Your move: PAPER
You: PAPER Computer: LIZARD
Computer wins round
Your move: SCISSORS
You: SCISSORS Computer: LIZARD
You win round
Your move: LIZARD
You: LIZARD Computer: LIZARD
Tie
Your move: SPOCK
You: SPOCK Computer: SPOCK
Tie
Computer wins 1-2

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>
```

Practical 2

Part A	
1)	Smart thermostat (Thermostat.java, main in this file) (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print “Already at maximum (30)”. (e) lower(): if temperature > MIN subtract 1, else print “Already at minimum (16)”. (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().
	Thermostat.java
Code	

```
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16, MAX = 30;
    private static int activeCount = 0;
    Thermostat(String location, int startTemp) {
        this.location = location;
        temperature = 22;
        if (startTemp >= MIN && startTemp <= MAX) { temperature = startTemp; }
        activeCount++;
    }
    Thermostat(String location) { this(location, 22); }
    void raise() {
        if (temperature < MAX) { temperature++; }
        else { System.out.println("Already at maximum (30)"); }
    }
    void lower() {
        if (temperature > MIN) { temperature--; }
        else { System.out.println("Already at minimum (16)"); }
    }
    int getTemperature() { return temperature; }
    static int getActiveCount() { return activeCount; }
    public static void main(String[] args) {
        Thermostat a = new Thermostat("Room 1");
        Thermostat b = new Thermostat("Room 2", 24);
        for (int i = 0; i < 10; i++) {
            a.raise(); System.out.println(a.getTemperature());
        }
        for (int i = 0; i < 20; i++) {
            a.lower(); System.out.println(a.getTemperature());
        }
        System.out.println("Active: " + getActiveCount());
    }
}
```

Output	

2)	Cinema show (CinemaShow.java, main in this file) (a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0. (b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100). (c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged. (d) cancel(int n): increase seatsAvailable by n but never above capacity. (e) Add getSeatsAvailable() and static getTotalBooked(). (f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked().

	CinemaShow.java

Code	public class CinemaShow {
	private String title;
	private int seatsAvailable;
	private final int capacity;
	private static int totalBooked = 0;
	CinemaShow(String title, int capacity) {
	this.title = title;
	this.capacity = capacity;
	seatsAvailable = capacity;
	}
	CinemaShow(String title) { this(title, 100); }
	boolean book(int n) {
	if (n > 0 && n <= seatsAvailable) {
	seatsAvailable -= n; totalBooked += n; return true;
	}
	return false;
	}
	void cancel(int n) {
	if (n > 0) { seatsAvailable = Math.min(capacity, seatsAvailable + n); }
	}
	int getSeatsAvailable() { return seatsAvailable; }
	static int getTotalBooked() { return totalBooked; }
	public static void main(String[] args) {
	CinemaShow s = new CinemaShow("Movie", 10);
	System.out.println(s.book(6) + " Seats: " + s.getSeatsAvailable());
	System.out.println(s.book(8) + " Seats: " + s.getSeatsAvailable());
	s.cancel(2);
	System.out.println("After cancel: " + s.getSeatsAvailable());
	System.out.println(s.book(3) + " Seats: " + s.getSeatsAvailable());
	System.out.println("Total booked: " + getTotalBooked());
	}
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB02" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>javac CinemaShow.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>java CinemaShow true Seats: 4 false Seats: 4 After cancel: 6 true Seats: 3 Total booked: 9 C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>

3)	(For Advanced Learners – home) Parking lot (ParkingLot.java) (a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0. (b) park(String type): for “two”, if twoWheelers < twoCap increment and add 20 to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject. (c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue.
	ParkingLot.java
Code	public class ParkingLot { private int twoWheelers, fourWheelers; private final int twoCap, fourCap; private static long revenue = 0;

```
    ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap; this.fourCap = fourCap;
    }
    void park(String type) {
        if (type.equals("two") && twoWheelers < twoCap) {
            twoWheelers++; revenue += 20; System.out.println("Two parked");
        } else if (type.equals("four") && fourWheelers < fourCap) {
            fourWheelers++; revenue += 40; System.out.println("Four parked");
        } else { System.out.println("Full or invalid type"); }
    }
    void leave(String type) {
        if (type.equals("two") && twoWheelers > 0) { twoWheelers--; }
        else if (type.equals("four") && fourWheelers > 0) { fourWheelers--; }
        System.out.println("Occupancy: " + twoWheelers + ", " + fourWheelers);
    }
    public static void main(String[] args) {
        ParkingLot p = new ParkingLot(1, 1);
        p.park("two"); p.park("two"); p.park("four");
        p.leave("two"); p.park("two");
        System.out.println("Final: " + p.twoWheelers + ", " + p.fourWheelers);
        System.out.println("Revenue: " + revenue);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB02"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>javac ParkingLot.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>java ParkingLot
Two parked
Full or invalid type
Four parked
Occupancy: 0, 1
Two parked
Final: 1, 1
Revenue: 80

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>
```

Practical 3

Part A

1) Distinct points (Point.java + Driver.java with main)
(a) Create class Point with private int x, y and a constructor.
(b) Override toString() to return “(x, y)”.
(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)).
(d) In Driver.main: build a Point[] with some repeated coordinates.
(e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not.
(f) Print “Distinct: N”.

Point.java

Code

```
import java.util.Objects;
public class Point {
    private int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    public String toString() { return "(" + x + ", " + y + ")"; }
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) { return false; }
        return x == p.x && y == p.y;
    }
    public int hashCode() { return Objects.hash(x, y); }
}
```

Driver.java

Code

```
public class Driver {
    public static void main(String[] args) {
        Point[] points = {new Point(1,2), new Point(3,4), new Point(1,2),
                          new Point(5,6), new Point(3,4)};
        int count = 0;
        for (int i = 0; i < points.length; i++) {
            boolean found = false;
            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) { found = true; break; }
            }
            if (!found) { count++; }
        }
        System.out.println("Distinct: " + count);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB03\Q1"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q1>javac Point.java Driver.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q1>java Driver
Distinct: 3

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q1>
```

2)	Duplicate card check (Card.java + Driver.java) (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: <card>" the first time a repeat appears.
	Card.java

Code	`import java.util.Objects;`
	`public class Card {`
	`  private String rank, suit;`
	`  Card(String rank, String suit) { this.rank = rank; this.suit = suit; }`
	`  public String toString() { return rank + " of " + suit; }`
	`  public boolean equals(Object o) {`
	`  if (!(o instanceof Card c)) { return false; }`
	`  return rank.equals(c.rank) && suit.equals(c.suit);`
	`  }`
	`  public int hashCode() { return Objects.hash(rank, suit); }`
	`}`

	Driver.java

Code	`public class Driver {`
	`  public static void main(String[] args) {`
	`  Card[] input = {new Card("Ace", "Spades"), new Card("Queen", "Hearts"),`
	`  new Card("Ace", "Spades")};`
	`  Card[] cards = new Card[input.length];`
	`  int size = 0;`
	`  for (Card c : input) {`
	`  boolean duplicate = false;`
	`  for (int i = 0; i < size; i++) {`
	`  if (c.equals(cards[i])) { duplicate = true; break; }`
	`  }`
	`  if (duplicate) {`
	`  System.out.println("Duplicate found: " + c); break;`
	`  }`
	`  cards[size++] = c;`
	`  }`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB03\Q2"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q2>javac Card.java Driver.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q2>java Driver`
	`Duplicate found: Ace of Spades`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q2>`

3)	(For Advanced Learners – home) Fractions (Fraction.java + Driver.java) (a) Create class Fraction with private int num, den. (b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g. (c) Override toString() to return num + "/" + den. (d) Override equals()/hashCode() using the reduced num and den. (e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.

	Fraction.java

Code	import java.util.Objects;
	public class Fraction {
	  private int num, den;
	  Fraction(int num, int den) {
	  if (den == 0) { throw new IllegalArgumentException("Zero denominator"); }
	  if (den < 0) { num = -num; den = -den; }
	  int g = gcd(Math.abs(num), den);
	  this.num = num / g; this.den = den / g;
	  }
	  private int gcd(int a, int b) {
	  while (b != 0) { int t = a % b; a = b; b = t; }
	  return a;
	  }
	  public String toString() { return num + "/" + den; }
	  public boolean equals(Object o) {
	  if (!(o instanceof Fraction f)) { return false; }
	  return num == f.num && den == f.den;
	  }
	  public int hashCode() { return Objects.hash(num, den); }
	}

	Driver.java

Code	public class Driver {
	  public static void main(String[] args) {
	  Fraction a = new Fraction(1, 2);
	  Fraction b = new Fraction(2, 4);
	  Fraction c = new Fraction(3, 6);
	  System.out.println(a + " " + b + " " + c);
	  System.out.println(a.equals(b) && b.equals(c));
	  }
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB03\Q3" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q3>javac Fraction.java Driver.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q3>java Driver 1/2 1/2 1/2 true C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q3>

Practical 4

Part A

1) Password strength (PasswordChecker.java + Driver.java)
(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.
(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4).
(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.

PasswordChecker.java

Code

```
public class PasswordChecker {
    static boolean[] rules(String pw) {
        return new boolean[]{pw.length() >= 8, pw.matches(".*[A-Z].*"),
            pw.matches(".*[0-9].*"), pw.matches(".*[^a-zA-Z0-9].*")};
    }
    static String strength(String pw) {
        int count = 0;
        for (boolean pass : rules(pw)) { if (pass) { count++; } }
        if (count <= 1) { return "Weak"; }
        if (count <= 3) { return "Medium"; }
        return "Strong";
    }
}
```

Driver.java

Code

```
import java.util.Arrays;
public class Driver {
    public static void main(String[] args) {
        String[] passwords = {"abc", "Abcd1234!", "abcd1234"};
        for (String pw : passwords) {
            System.out.println(pw);
            System.out.println("Length, uppercase, digit, special: "
                + Arrays.toString(PasswordChecker.rules(pw)));
            System.out.println(PasswordChecker.strength(pw));
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB04\Q1"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q1>javac PasswordChecker.java Driver.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q1>java Driver
abc
Length, uppercase, digit, special: [false, false, false, false]
Weak
Abcd1234!
Length, uppercase, digit, special: [true, true, true, true]
Strong
abcd1234
Length, uppercase, digit, special: [true, false, true, false]
Medium

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q1>
```

2)	Chat-log filter (ChatFilter.java + Driver.java) (a) In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts). (b) Read a keyword from the user. (c) For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts. (d) If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder. (e) Print “Matches: N” and the built report.
	ChatFilter.java

Code	`public class ChatFilter {`
	`  static String filter(String[] lines, String keyword) {`
	`  int count = 0;`
	`  StringBuilder report = new StringBuilder();`
	`  for (String line : lines) {`
	`  String[] p = line.split(" ", 3);`
	`  if (p.length != 3) { continue; }`
	`  if (p[2].toLowerCase().contains(keyword.toLowerCase())) {`
	`  count++;`
	`  report.append(p[0]).append(" ").append(p[1]).append(": ")`
	`  .append(p[2]).append("\n");`
	`  }`
	`  }`
	`  return "Matches: " + count + "\n" + report;`
	`  }`
	`}`

	Driver.java

Code	`import java.util.Scanner;`
	`public class Driver {`
	`  public static void main(String[] args) {`
	`  String[] lines = {"10:05 alice Hello there", "10:06 bob Good morning", "bad"};`
	`  System.out.print("Keyword: ");`
	`  String keyword = new Scanner(System.in).nextLine();`
	`  System.out.print(ChatFilter.filter(lines, keyword));`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB04\Q2"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q2>javac ChatFilter.java Driver.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q2>java Driver`
	`Keyword: hello`
	`Matches: 1`
	`10:05 alice: Hello there`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q2>`

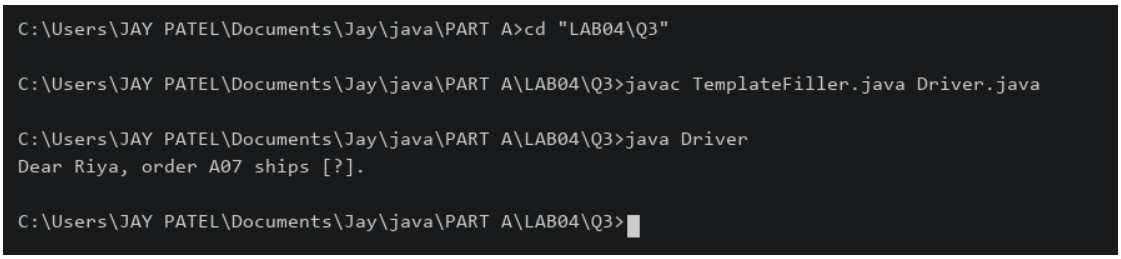
3)	(For Advanced Learners – home) Template filler (TemplateFiller.java + Driver.java) (a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date} is not supplied). (b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template. (c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found. (d) Print the filled template.

	TemplateFiller.java

Code	import java.util.regex.*;
	public class TemplateFiller {
	  static String fill(String template, String[] names, String[] values) {
	  Matcher m = Pattern.compile("\\{(\\w+)\\}").matcher(template);
	  StringBuilder result = new StringBuilder();
	  while (m.find()) {
	  String value = "[?]";
	  for (int i = 0; i < names.length; i++) {
	  if (names[i].equals(m.group(1))) { value = values[i]; break; }
	  }
	  m.appendReplacement(result, Matcher.quoteReplacement(value));
	  }
	  m.appendTail(result);
	  return result.toString();
	  }
	}

	Driver.java

Code	public class Driver {
	  public static void main(String[] args) {
	  String template = "Dear {name}, order {id} ships {date}.";
	  String[] names = {"name", "id"};
	  String[] values = {"Riya", "A07"};
	  System.out.println(TemplateFiller.fill(template, names, values));
	  }
	}

Output	

Practical 5

Part A

1) Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism.

ShapeDemo.java

Code

```
abstract class Shape { abstract double area(); }
class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }
    double area() { return Math.PI * radius * radius; }
}
class Rectangle extends Shape {
    double length, width;
    Rectangle(double length, double width) { this.length = length; this.width = width; }
    double area() { return length * width; }
}
class Triangle extends Shape {
    double base, height;
    Triangle(double base, double height) { this.base = base; this.height = height; }
    double area() { return base * height / 2; }
}
public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] shapes = {new Circle(2), new Rectangle(4,5), new Triangle(3,6)};
        double total = 0, largest = -1;
        String name = "";
        for (Shape s : shapes) {
            double area = s.area(); total += area;
            System.out.println(s.getClass().getSimpleName() + ": " + area
                + " Running total: " + total);
            if (area > largest) { largest = area; name = s.getClass().getSimpleName(); }
        }
        System.out.println("Largest: " + name + " " + largest);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB05"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>javac ShapeDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>java ShapeDemo
Circle: 12.566370614359172 Running total: 12.566370614359172
Rectangle: 20.0 Running total: 32.56637061435917
Triangle: 9.0 Running total: 41.56637061435917
Largest: Rectangle 20.0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>
```

2) Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.

Payroll.java

Code

```
abstract class Employee {
    String name, id;
    Employee(String name, String id) { this.name = name; this.id = id; }
    abstract double monthlySalary();
```

```
}
class FullTime extends Employee {
    double salary;
    FullTime(String n, String id, double salary) { super(n,id); this.salary = salary; }
    double monthlySalary() { return salary; }
}
class PartTime extends Employee {
    int hours; double rate;
    PartTime(String n, String id, int hours, double rate) {
        super(n,id); this.hours = hours; this.rate = rate;
    }
    double monthlySalary() { return hours * rate; }
}
class Intern extends Employee {
    double stipend;
    Intern(String n, String id, double stipend) { super(n,id); this.stipend = stipend; }
    double monthlySalary() { return stipend; }
}
public class Payroll {
    public static void main(String[] args) {
        Employee[] staff = {new FullTime("Tej","E1",30000),
            new PartTime("Riya","E2",40,200), new Intern("Aman","E3",5000)};
        double total = 0;
        for (Employee e : staff) {
            System.out.println(e.id + " " + e.name + " " + e.monthlySalary());
            total += e.monthlySalary();
            if (e instanceof Intern) { System.out.println("Intern stipend"); }
        }
        System.out.println("Total: " + total);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB05"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>javac Payroll.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>java Payroll
E1 Tej 30000.0
E2 Riya 8000.0
E3 Aman 5000.0
Intern stipend
Total: 43000.0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>
```

3) (For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself.

MediaDemo.java

Code

```
abstract class Media {
    String title; int daysLate;
    Media(String title, int daysLate) { this.title = title; this.daysLate = daysLate; }
    abstract double lateFee();
}
class Book extends Media {
    Book(String title, int days) { super(title, days); }
    double lateFee() { return Math.max(0, daysLate) * 2; }
}
class DVD extends Media {
    DVD(String title, int days) { super(title, days); }
    double lateFee() { return Math.max(0, daysLate) * 5; }
}
class Magazine extends Media {
    Magazine(String title, int days) { super(title, days); }
    double lateFee() { return Math.max(0, daysLate - 1); }
```

```
}
public class MediaDemo {
    public static void main(String[] args) {
        Media[] items = {new Book("Java",3), new DVD("Movie",2),
                         new Magazine("Science",4)};
        double total = 0;
        for (Media m : items) {
            System.out.println(m.title + ": " + m.lateFee()); total += m.lateFee();
        }
        System.out.println("Total fees: " + total);
    }
}
```

Rules: Book Rs 2/day; DVD Rs 5/day; Magazine one free day, then Rs 1/day.

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB05"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>javac MediaDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>java MediaDemo
Java: 6.0
Movie: 10.0
Science: 3.0
Total fees: 19.0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>
```

Practical 6

Part A	
1)	Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.
	RemoteDemo.java

Code	interface Switchable {
	  void on();
	  void off();
	  default void toggle() { on(); off(); }
	}
	class Fan implements Switchable {
	  public void on() { System.out.println("Fan ON"); }
	  public void off() { System.out.println("Fan OFF"); }
	}
	class Light implements Switchable {
	  public void on() { System.out.println("Light ON"); }
	  public void off() { System.out.println("Light OFF"); }
	}
	@FunctionalInterface
	interface Permission { boolean allow(Switchable device, int hour); }
	public class RemoteDemo {
	  public static void main(String[] args) {
	  Switchable[] devices = {new Fan(), new Light()};
	  for (Switchable d : devices) { d.toggle(); }
	  Permission p1 = new Permission() {
	  public boolean allow(Switchable d, int hour) { return hour < 22; }
	  };
	  Permission p2 = (d, hour) -> hour < 22;
	  System.out.println(p1.allow(devices[0], 10));
	  System.out.println(p2.allow(devices[0], 10));
	  }
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB06" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>javac RemoteDemo.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>java RemoteDemo Fan ON Fan OFF Light ON Light OFF true true C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>

2)	Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.
	NotificationDemo.java
Code	@FunctionalInterface interface Notifier { void send(String message); } interface Urgent {} public class NotificationDemo {

```
    public static void main(String[] args) {
        Notifier email = message -> System.out.println("Email: " + message);
        Notifier sms = (Notifier & Urgent) message -> System.out.println("SMS: " + message);
        Notifier[] senders = {email, sms};
        for (Notifier sender : senders) {
            sender.send("Meeting at 10 AM");
            if (sender instanceof Urgent) { sender.send("Meeting at 10 AM"); }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB06"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>javac NotificationDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>java NotificationDemo
Email: Meeting at 10 AM
SMS: Meeting at 10 AM
SMS: Meeting at 10 AM

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>
```

3) (For Advanced Learners – home) Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.

rules/Discount.java

Code

```
package rules;
@FunctionalInterface
public interface Discount { double apply(double price); }
```

app/DiscountApp.java

Code

```
package app;
import java.util.Scanner;
import rules.Discount;
public class DiscountApp {
    public static void main(String[] args) {
        double[] prices = {100, 200, 500};
        Discount[] rules = {p -> p, p -> p * 0.9, p -> p * 0.8};
        System.out.println("1. No discount\n2. 10% discount\n3. 20% discount");
        int choice = new Scanner(System.in).nextInt();
        if (choice < 1 || choice > 3) { System.out.println("Invalid choice"); return; }
        Discount selected = rules[choice - 1];
        for (double p : prices) { System.out.println(p + " -> " + selected.apply(p)); }
    }
}
```

Build and run from the folder containing app and rules:
javac -d out rules/Discount.java app/DiscountApp.java
jar --create --file discount.jar --main-class app.DiscountApp -C out .
java -jar discount.jar

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB06\Q3"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>javac -d out rules\Discount.java app\DiscountApp.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>jar --create --file discount.jar --main-class app.DiscountApp -C out .

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>java -jar discount.jar
1. No discount
2. 10% discount
3. 20% discount
2
100.0 -> 90.0
200.0 -> 180.0
500.0 -> 450.0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>
```

Practical 7

Part A	
1)	Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.
	FormDemo.java
Code	

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {}
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength { int value(); }
class SignupForm {
    @NotBlank @MaxLength(10) String name;
    @NotBlank String email;
    SignupForm(String name, String email) { this.name = name; this.email = email; }
}
public class FormDemo {
    static ArrayList<String> check(Object obj) throws IllegalAccessException {
        ArrayList<String> errors = new ArrayList<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object value = f.get(obj);
            if (f.isAnnotationPresent(NotBlank.class)
                && (value == null || value.toString().isBlank())) {
                errors.add(f.getName() + " must not be blank");
            }
            MaxLength rule = f.getAnnotation(MaxLength.class);
            if (rule != null && value != null && value.toString().length() > rule.value()) {
                errors.add(f.getName() + " is too long");
            }
        }
        return errors;
    }
    public static void main(String[] args) throws Exception {
        System.out.println(check(new SignupForm("VeryLongUserName", "")));
        System.out.println(check(new SignupForm("Tej", "tej@test.com")));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB07"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>javac FormDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>java FormDemo
[name is too long, email must not be blank]
[]

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>
```

2)	Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.
	MiniRunner.java
Code	import java.lang.annotation.*;

```
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {}
class Checks {
   @Run public void first() { System.out.println("First ran"); }
   @Run public void second() { System.out.println("Second ran"); }
   public void helper() { System.out.println("Helper"); }
}
public class MiniRunner {
   public static void main(String[] args) throws Exception {
      Checks checks = new Checks();
      int count = 0;
      for (Method m : Checks.class.getDeclaredMethods()) {
         if (m.isAnnotationPresent(Run.class) && m.getParameterCount() == 0) {
            m.setAccessible(true);
            try { m.invoke(checks); }
            catch (InvocationTargetException e) {
               System.out.println("Failed: " + e.getCause().getMessage());
            }
            count++;
         }
      }
      System.out.println("Methods ran: " + count);
   }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB07"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>javac MiniRunner.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>java MiniRunner
First ran
Second ran
Methods ran: 2

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>
```

3) (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

ColumnDemo.java

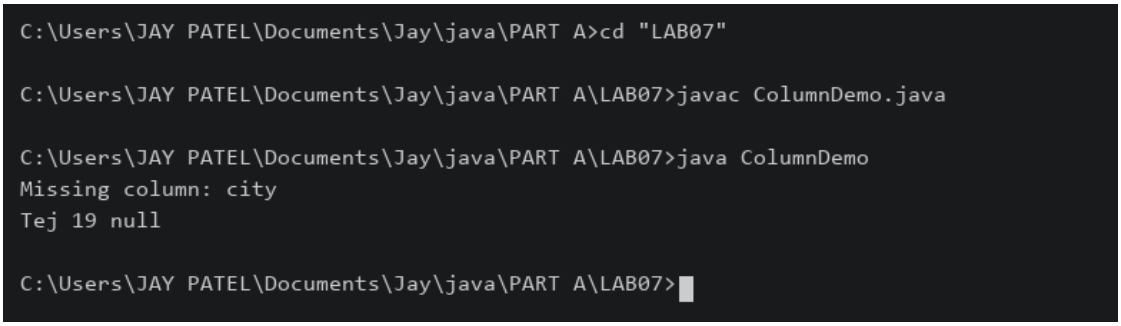
Code

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column { String name(); }
class Student {
   @Column(name="name") String name;
   @Column(name="age") int age;
   @Column(name="city") String city;
}
public class ColumnDemo {
   static void populate(Object obj, String[] headers, String[] row) throws Exception {
      for (Field f : obj.getClass().getDeclaredFields()) {
         Column column = f.getAnnotation(Column.class);
         if (column == null) { continue; }
         int index = -1;
         for (int i = 0; i < headers.length; i++) {
            if (headers[i].equals(column.name())) { index = i; break; }
         }
         if (index < 0 || index >= row.length) {
            System.out.println("Missing column: " + column.name()); continue;
         }
```

```
            f.setAccessible(true);
            if (f.getType() == int.class) { f.setInt(obj, Integer.parseInt(row[index])); }
            else { f.set(obj, row[index]); }
        }
    }
    public static void main(String[] args) throws Exception {
        Student s = new Student();
        populate(s, new String[]{"name", "age"}, new String[]{"Tej", "19"});
        System.out.println(s.name + " " + s.age + " " + s.city);
    }
}
```

A missing column is reported and the field keeps its default value.

Output



Practical 8

Part A

1) Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

Calculator.java

Code

```
import java.util.Scanner;
class DivideByZeroException extends Exception {
    DivideByZeroException() { super("Cannot divide by zero"); }
}
public class Calculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean done = false;
        while (!done) {
            try {
                System.out.print("First number: ");
                double a = Double.parseDouble(sc.nextLine());
                System.out.print("Second number: ");
                double b = Double.parseDouble(sc.nextLine());
                System.out.print("Operator: ");
                String op = sc.nextLine();
                double result;
                switch (op) {
                    case "+" -> result = a + b;
                    case "-" -> result = a - b;
                    case "*" -> result = a * b;
                    case "/" -> {
                        if (b == 0) { throw new DivideByZeroException(); }
                        result = a / b;
                    }
                    default -> throw new IllegalArgumentException("Invalid operator");
                }
                System.out.println("Result: " + result); done = true;
            } catch (NumberFormatException e) { System.out.println("Invalid number"); }
            catch (DivideByZeroException e) { System.out.println(e.getMessage()); }
            catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
            finally { System.out.println("Attempt logged"); }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB08"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>javac Calculator.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>java Calculator
First number: 10
Second number: 0
Operator: /
Cannot divide by zero
Attempt logged
First number: 15
Second number: 3
Operator: /
Result: 5.0
Attempt logged

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>
```

2)	Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.
	StockDemo.java
Code	

```
import java.util.HashMap;
class OutOfStockException extends Exception {
    private int shortfall;
    OutOfStockException(int shortfall) {
        super("Short by " + shortfall); this.shortfall = shortfall;
    }
    int getShortfall() { return shortfall; }
}
class InvalidQuantityException extends Exception {
    InvalidQuantityException() { super("Quantity must be positive"); }
}
class Warehouse {
    HashMap<String,Integer> stock = new HashMap<>();
    Warehouse() { stock.put("Pen", 10); stock.put("Book", 5); }
    void issue(String item, int qty)
        throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0) { throw new InvalidQuantityException(); }
        int available = stock.getOrDefault(item, 0);
        if (qty > available) { throw new OutOfStockException(qty - available); }
        stock.put(item, available - qty);
        System.out.println("Issued " + qty + " " + item);
    }
}
public class StockDemo {
    public static void main(String[] args) {
        Warehouse w = new Warehouse();
        String[] items = {"Pen", "Book", "Pen", "Book"};
        int[] qty = {4, 8, 0, 2};
        for (int i = 0; i < items.length; i++) {
            try { w.issue(items[i], qty[i]); }
            catch (OutOfStockException e) { System.out.println(e.getMessage()); }
            catch (InvalidQuantityException e) { System.out.println(e.getMessage()); }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB08"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>javac StockDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>java StockDemo
Issued 4 Pen
Short by 3
Quantity must be positive
Issued 2 Book

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>
```

3)	(For Advanced Learners – home) Create an AutoCloseable resource and use try-with-resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource.
	ResourceDemo.java
Code	

```
class ReportSession implements AutoCloseable {
    public void close() throws Exception {
        System.out.println("Session closed");
        throw new Exception("Close error");
    }
}
```

```
public class ResourceDemo {
    public static void main(String[] args) {
        try (ReportSession session = new ReportSession()) {
            System.out.println("Session opened");
            throw new Exception("Original processing error");
        } catch (Exception e) {
            System.out.println("Reported: " + e.getMessage());
            for (Throwable error : e.getSuppressed()) {
                System.out.println("Suppressed: " + error.getMessage());
            }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB08"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>javac ResourceDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>java ResourceDemo
Session opened
Session closed
Reported: Original processing error
Suppressed: Close error

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>
```

Practical 9

Part A	
1)	Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.
	CounterRace.java

Code	`public class CounterRace {`
	`  static class Counter {`
	`  int value;`
	`  void unsafe() { value++; }`
	`  synchronized void safe() { value++; }`
	`  }`
	`  static void run(boolean safe) throws InterruptedException {`
	`  Counter counter = new Counter();`
	`  Thread[] threads = new Thread[10];`
	`  for (int i = 0; i < threads.length; i++) {`
	`  threads[i] = new Thread(() -> {`
	`  for (int j = 0; j < 100000; j++) {`
	`  if (safe) { counter.safe(); } else { counter.unsafe(); }`
	`  }`
	`  });`
	`  threads[i].start();`
	`  }`
	`  for (Thread t : threads) { t.join(); }`
	`  System.out.println("Synchronized: " + safe + " Total: " + counter.value);`
	`  }`
	`  public static void main(String[] args) throws Exception { run(false); run(true); }`
	`}`

	Race results vary; an unsafe run can occasionally produce the expected total.

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB09"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>javac CounterRace.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>java CounterRace`
	`Synchronized: false Total: 145105`
	`Synchronized: true Total: 1000000`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>`

2)	Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.
	SeatRace.java

Code	`public class SeatRace {`
	`  static class Booking {`
	`  int seatsLeft = 5;`
	`  boolean unsafe() {`
	`  if (seatsLeft > 0) {`
	`  try { Thread.sleep(10); }`
	`  catch (InterruptedException e) { Thread.currentThread().interrupt(); return false; }`
	`  seatsLeft--; return true;`
	`  }`
	`  return false;`
	`  }`
	`  synchronized boolean book() {`
	`  if (seatsLeft > 0) { seatsLeft--; return true; }`

```
            return false;
        }
    }
    static void run(boolean safe) throws Exception {
        Booking booking = new Booking();
        Thread[] threads = new Thread[10];
        boolean[] success = new boolean[10];
        for (int i = 0; i < 10; i++) {
            final int id = i;
            threads[i] = new Thread(() -> {
                if (safe) { success[id] = booking.book(); }
                else { success[id] = booking.unsafe(); }
            });
            threads[i].start();
        }
        for (Thread t : threads) { t.join(); }
        int count = 0;
        for (boolean booked : success) { if (booked) { count++; } }
        System.out.println("Synchronized: " + safe + " Booked: " + count);
    }
    public static void main(String[] args) throws Exception { run(false); run(true); }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB09"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>javac SeatRace.java

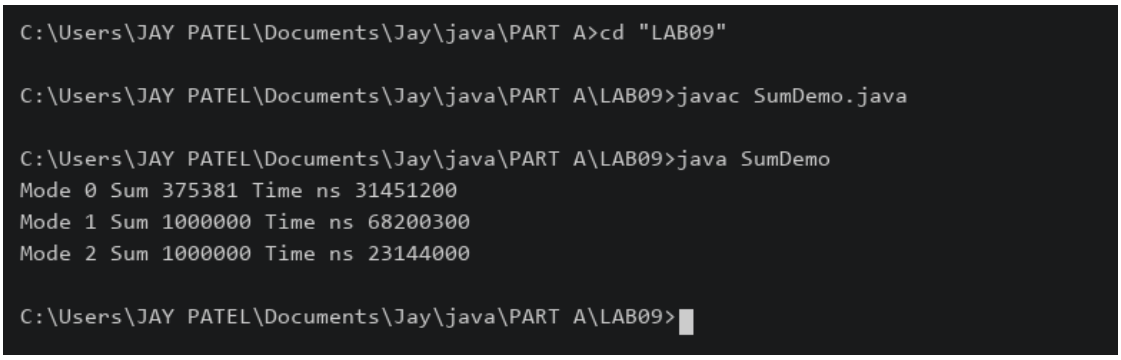
C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>java SeatRace
Synchronized: false Booked: 10
Synchronized: true Booked: 5

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>
```

3)	(For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference.
	SumDemo.java
Code	

```
import java.util.concurrent.atomic.AtomicLong;
public class SumDemo {
    static long total;
    static synchronized void add(long n) { total += n; }
    static void run(int mode, int[] values) throws Exception {
        total = 0;
        AtomicLong atomic = new AtomicLong();
        Thread[] threads = new Thread[4];
        long start = System.nanoTime();
        for (int i = 0; i < 4; i++) {
            int from = i * values.length / 4;
            int to = (i + 1) * values.length / 4;
            threads[i] = new Thread(() -> {
                for (int j = from; j < to; j++) {
                    if (mode == 0) { total += values[j]; }
                    else if (mode == 1) { add(values[j]); }
                    else { atomic.addAndGet(values[j]); }
                }
            });
            threads[i].start();
        }
        for (Thread t : threads) { t.join(); }
        long result = total;
        if (mode == 2) { result = atomic.get(); }
        System.out.println("Mode " + mode + " Sum " + result
            + " Time ns " + (System.nanoTime() - start));
    }
```

	``
	    public static void main(String[] args) throws Exception {
	        int[] values = new int[1000000];
	        java.util.Arrays.fill(values, 1);
	        run(0, values); run(1, values); run(2, values);
	    }
	}
	``

	Mode 0 is unsafe; mode 1 uses synchronization; mode 2 uses AtomicLong. Times include thread creation and are illustrative.
Output	

Practical 10

Part A	
1)	Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.
	PoolDemo.java

Code	`import java.util.concurrent.*;`
	`public class PoolDemo {`
	`  public static void main(String[] args) throws Exception {`
	`  ExecutorService pool = Executors.newFixedThreadPool(3);`
	`  for (int i = 1; i <= 10; i++) {`
	`  final int id = i;`
	`  pool.execute(() -> {`
	`  System.out.println("Task " + id + " " + Thread.currentThread().getName());`
	`  try { Thread.sleep(100); }`
	`  catch (InterruptedException e) { Thread.currentThread().interrupt(); }`
	`  });`
	`  }`
	`  pool.shutdown();`
	`  if (!pool.awaitTermination(1, TimeUnit.MINUTES)) { pool.shutdownNow(); }`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB10"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>javac PoolDemo.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>java PoolDemo`
	`Task 2 pool-1-thread-2`
	`Task 3 pool-1-thread-3`
	`Task 1 pool-1-thread-1`
	`Task 5 pool-1-thread-1`
	`Task 6 pool-1-thread-3`
	`Task 4 pool-1-thread-2`
	`Task 7 pool-1-thread-3`
	`Task 8 pool-1-thread-1`
	`Task 9 pool-1-thread-2`
	`Task 10 pool-1-thread-2`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>`

2)	Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.
	BufferDemo.java

Code	`class Buffer {`
	`  private int value;`
	`  private boolean full;`
	`  synchronized void put(int n) throws InterruptedException {`
	`  while (full) { wait(); }`
	`  value = n; full = true;`
	`  System.out.println("Produced: " + n); notify();`
	`  }`
	`  synchronized int take() throws InterruptedException {`
	`  while (!full) { wait(); }`
	`  int n = value; full = false;`
	`  System.out.println("Consumed: " + n); notify(); return n;`

```
    }
}
public class BufferDemo {
    public static void main(String[] args) throws Exception {
        Buffer buffer = new Buffer();
        Thread producer = new Thread(() -> {
            try { for (int i = 1; i <= 5; i++) { buffer.put(i); } }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread consumer = new Thread(() -> {
            try { for (int i = 1; i <= 5; i++) { buffer.take(); } }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        producer.start(); consumer.start(); producer.join(); consumer.join();
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB10"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>javac BufferDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>java BufferDemo
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>
```

3) (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself.

LockDemo.java

Code

```
import java.util.concurrent.CountDownLatch;
public class LockDemo {
    static void pause() {
        try { Thread.sleep(50); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
    static void lock(Object a, Object b) {
        synchronized (a) { pause(); synchronized (b) { System.out.println("Completed"); } }
    }
    public static void main(String[] args) throws Exception {
        Object a = new Object(), b = new Object();
        if (args.length > 0 && args[0].equals("deadlock")) {
            CountDownLatch held = new CountDownLatch(2);
            Runnable first = () -> {
                synchronized (a) {
                    held.countDown();
                    try { held.await(); } catch (InterruptedException e) { return; }
                    synchronized (b) { System.out.println("A then B"); }
                }
            };
            Runnable second = () -> {
                synchronized (b) {
                    held.countDown();
                    try { held.await(); } catch (InterruptedException e) { return; }
```

```
                synchronized (a) { System.out.println("B then A"); }
            }
        };
        new Thread(first).start(); new Thread(second).start();
    } else {
        Thread t1 = new Thread(() -> lock(a,b));
        Thread t2 = new Thread(() -> lock(a,b));
        t1.start(); t2.start(); t1.join(); t2.join();
    }
  }
}
```

java LockDemo deadlock deliberately hangs; stop it with Ctrl+C. java LockDemo runs the fixed version.

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB10"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>javac LockDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>java LockDemo
Completed
Completed

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>
```

Practical 11

Part A	
1)	Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.
	SaveDemo.java

Code	import java.io.*;
	class Student implements Serializable {
	private static final long serialVersionUID = 1L;
	String name;
	int marks;
	transient String password;
	Student(String name, int marks, String password) {
	this.name = name; this.marks = marks; this.password = password;
	}
	}
	public class SaveDemo {
	public static void main(String[] args) throws Exception {
	Student[] students = {new Student("Tej",90,"abc"), new Student("Riya",85,"xyz")};
	try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("students.dat")))
	{
	out.writeObject(students);
	}
	Student[] loaded;
	try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("students.dat"))) {
	loaded = (Student[]) in.readObject();
	}
	for (Student s : loaded) {
	System.out.println(s.name + " " + s.marks + " Password: " + s.password);
	}
	System.out.println("Fresh object: " + (students[0] != loaded[0]));
	}
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB11" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>javac SaveDemo.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>java SaveDemo Tej 90 Password: null Riya 85 Password: null Fresh object: true C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>

2)	Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.
	LogAnalyzer.java

Code	import java.nio.file.*;
	import java.nio.file.attribute.BasicFileAttributes;
	import java.io.BufferedReader;
	public class LogAnalyzer {
	public static void main(String[] args) throws Exception {
	Path folder = Path.of("samplelogs"); Files.createDirectories(folder);
	Files.writeString(folder.resolve("one.log"), "login OK\nlogin ERROR\n");
	Files.writeString(folder.resolve("two.log"), "payment ERROR\n");
	String keyword = "ERROR";

```
        int lines = 0, matches = 0;
        try (DirectoryStream<Path> files = Files.newDirectoryStream(folder, "*.log")) {
            for (Path file : files) {
                BasicFileAttributes attr = Files.readAttributes(file, BasicFileAttributes.class);
                System.out.println(file + " Size: " + attr.size());
                try (BufferedReader reader = Files.newBufferedReader(file)) {
                    String line;
                    while ((line = reader.readLine()) != null) {
                        lines++;
                        if (line.contains(keyword)) { matches++; }
                    }
                }
            }
        }
        System.out.println("Lines: " + lines + " Matches: " + matches);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB11"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>javac LogAnalyzer.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>java LogAnalyzer
samplelogs\one.log Size: 21
samplelogs\two.log Size: 14
Lines: 3 Matches: 2

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>
```

3) (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

TreeReport.java

Code

```
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.stream.Stream;
public class TreeReport {
    public static void main(String[] args) throws Exception {
        Path root = Path.of("sampletree"); Files.createDirectories(root.resolve("sub"));
        Files.writeString(root.resolve("sub/a.txt"), "Hello");
        Path report = Path.of("tree-report.txt");
        StringBuilder text = new StringBuilder();
        try (Stream<Path> paths = Files.walk(root)) {
            for (Path file : paths.filter(Files::isRegularFile).toList()) {
                BasicFileAttributes a = Files.readAttributes(file, BasicFileAttributes.class);
                text.append(root.relativize(file)).append(" ").append(a.size())
                    .append(" ").append(a.lastModifiedTime()).append("\n");
            }
        }
        Files.writeString(report, text); System.out.print(text);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB11"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>javac TreeReport.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>java TreeReport
sub\a.txt 5 2026-10-28T15:56:20.4590231Z

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>
```

Practical 12

Part A

1) Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.

WordFrequency.java

Code

```
import java.util.*;
public class WordFrequency {
    public static void main(String[] args) {
        String paragraph = "Java is simple. Java is useful. Java supports objects.";
        HashMap<String,Integer> counts = new HashMap<>();
        for (String word : paragraph.toLowerCase().split("[^a-z]+")) {
            if (!word.isEmpty()) { counts.put(word, counts.getOrDefault(word,0) + 1); }
        }
        ArrayList<Map.Entry<String,Integer>> list = new ArrayList<>(counts.entrySet());
        list.sort(Comparator.<Map.Entry<String,Integer>>comparingInt(e -> e.getValue())
            .reversed().thenComparing(e -> e.getKey()));
        for (Map.Entry<String,Integer> e : list) {
            System.out.println(e.getKey() + ": " + e.getValue());
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB12"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB12>javac WordFrequency.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB12>java WordFrequency
java: 3
is: 2
objects: 1
simple: 1
supports: 1
useful: 1

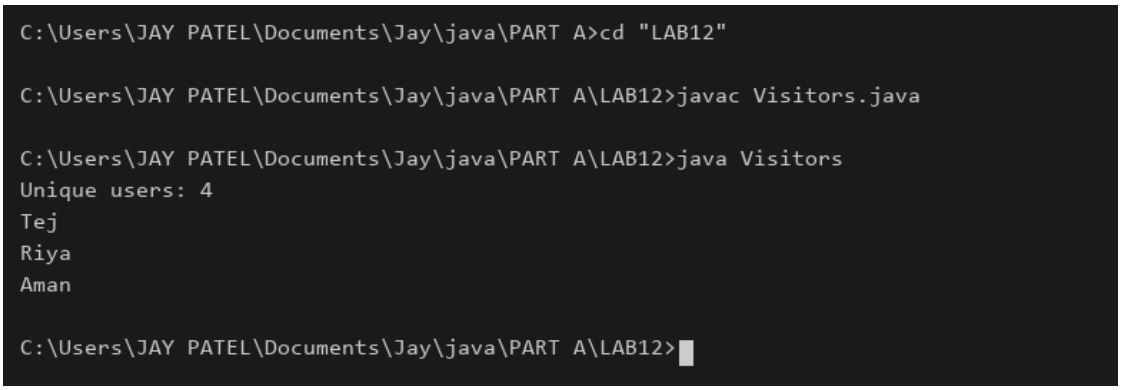
C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB12>
```

2) Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

Visitors.java

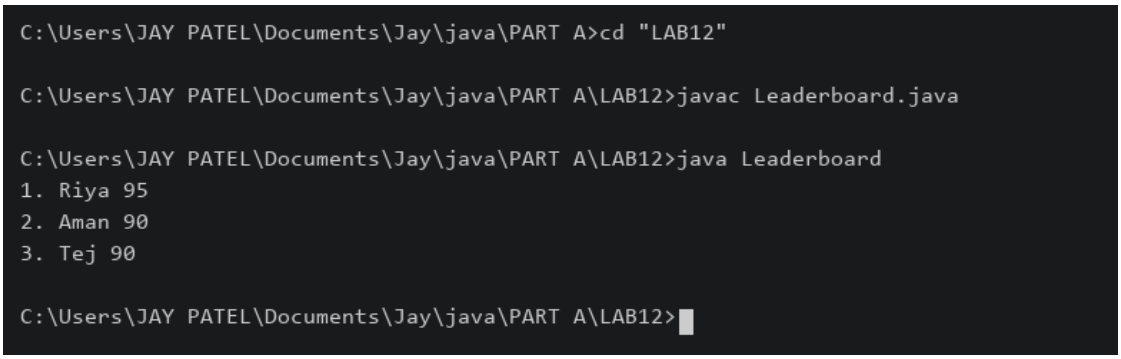
Code

```
import java.util.*;
public class Visitors {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Tej","Riya","Tej","Aman","Riya","Neha");
        HashSet<String> unique = new HashSet<>(names);
        LinkedHashSet<String> ordered = new LinkedHashSet<>(names);
        System.out.println("Unique users: " + unique.size());
        int count = 0;
        for (String name : ordered) {
            if (count == 3) { break; }
            System.out.println(name); count++;
        }
    }
}
```

Output	

3)	(For Advanced Learners – home) Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details.
	Leaderboard.java

Code	import java.util.*;
	public class Leaderboard {
	public static void main(String[] args) {
	Map<String,Integer> scores = new HashMap<>();
	scores.put("Tej",90); scores.put("Riya",95);
	scores.put("Aman",90); scores.put("Neha",80);
	ArrayList<Map.Entry<String,Integer>> list = new ArrayList<>(scores.entrySet());
	list.sort(Comparator.<Map.Entry<String,Integer>>comparingInt(e -> e.getValue())
	.reversed().thenComparing(e -> e.getKey()));
	int n = 3;
	for (int i = 0; i < Math.min(n,list.size()); i++) {
	System.out.println((i + 1) + ". " + list.get(i).getKey()
	+ " " + list.get(i).getValue());
	}
	}
	}

Output	

Practical 13

Part A	
1)	Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.
	ContainerDemo.java
Code	

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}
class Pair<A,B> {
    private A first; private B second;
    Pair(A first, B second) { this.first = first; this.second = second; }
    A getFirst() { return first; }
    B getSecond() { return second; }
}
public class ContainerDemo {
    static <T> T first(T[] values) {
        if (values.length == 0) { throw new IllegalArgumentException("Empty array"); }
        return values[0];
    }
    public static void main(String[] args) {
        Box<String> names = new Box<>(); names.set("Tej");
        Box<Integer> marks = new Box<>(); marks.set(90);
        Pair<String,Integer> pair = new Pair<>(names.get(), marks.get());
        System.out.println(pair.getFirst() + " " + pair.getSecond());
        System.out.println(first(new String[]{"Java","Python"}));
        System.out.println(first(new Integer[]{10,20}));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB13"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>javac ContainerDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>java ContainerDemo
Tej 90
Java
10

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>
```

2)	Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.
	Stats.java
Code	

```
public class Stats {
    static <N extends Number> double average(N[] values) {
        if (values.length == 0) { throw new IllegalArgumentException("Empty array"); }
        double sum = 0;
        for (N n : values) { sum += n.doubleValue(); }
        return sum / values.length;
    }
    public static void main(String[] args) {
        System.out.println(average(new Integer[]{10,20,30}));
        System.out.println(average(new Double[]{1.5,2.5}));
        System.out.println(average(new Long[]{100L,200L}));
    }
```

	}
	String[] is rejected at compile time because String does not extend Number.
Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB13" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>javac Stats.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>java Stats 20.0 2.0 150.0 C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>

3)	(For Advanced Learners – home) Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.
	StackDemo.java

Code	import java.util.EmptyStackException;
	class Stack<T> {
	private Object[] data;
	private int size;
	Stack(int capacity) { data = new Object[capacity]; }
	void push(T item) {
	if (size == data.length) { throw new IllegalStateException("Stack full"); }
	data[size++] = item;
	}
	@SuppressWarnings("unchecked")
	T pop() {
	if (size == 0) { throw new EmptyStackException(); }
	T item = (T) data[--size]; data[size] = null; return item;
	}
	}
	public class StackDemo {
	public static void main(String[] args) {
	Stack<Integer> numbers = new Stack<>(3); numbers.push(10); numbers.push(20);
	Stack<String> names = new Stack<>(2); names.push("Tej");
	System.out.println(numbers.pop()); System.out.println(names.pop());
	try { names.pop(); }
	catch (EmptyStackException e) { System.out.println("Stack empty"); }
	}
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB13" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>javac StackDemo.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>java StackDemo 20 Tej Stack empty C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>

Practical 14

Part A	
1)	Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.
	Item.java
Code	public record Item(String name) {}
	ItemService.java

Code	import java.util.ArrayList;
	public class ItemService {
	private ArrayList<Item> items = new ArrayList<>();
	public void add(String name) {
	if (name == null || name.isBlank()) { throw new IllegalArgumentException("Name required"); }
	items.add(new Item(name));
	}
	public int count() { return items.size(); }
	public ArrayList<Item> all() { return new ArrayList<>(items); }
	}

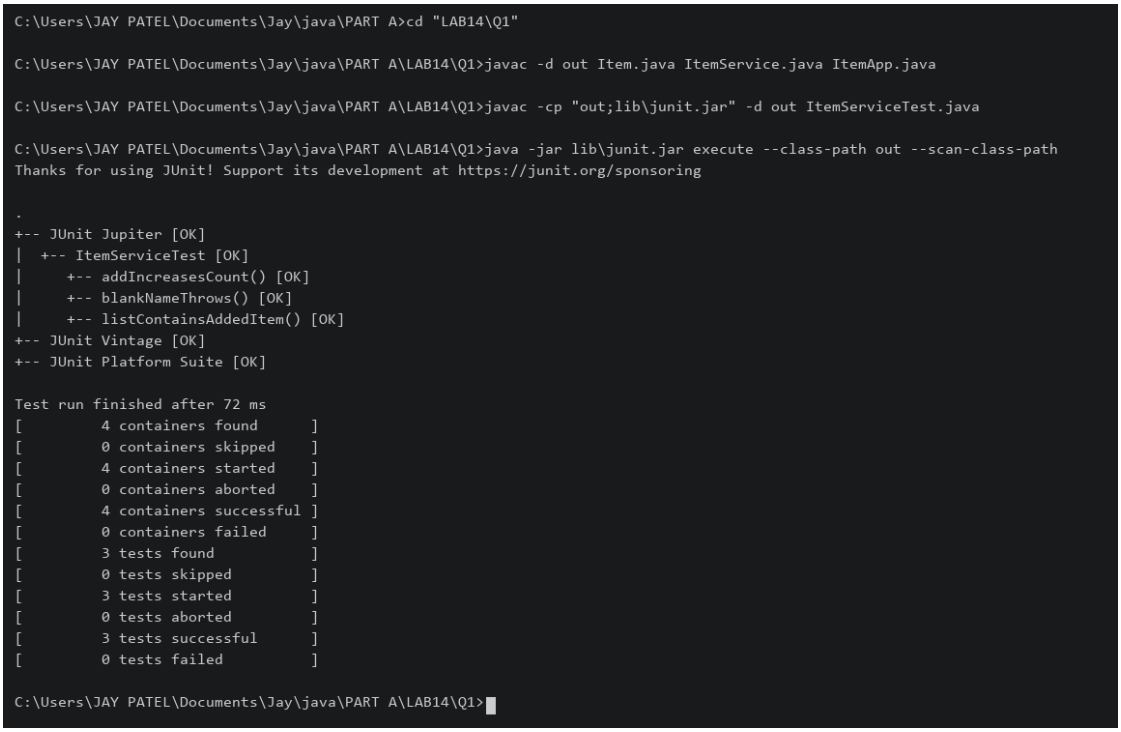
	ItemApp.java

Code	import java.util.Scanner;
	public class ItemApp {
	public static void main(String[] args) {
	ItemService service = new ItemService();
	Scanner sc = new Scanner(System.in);
	while (true) {
	System.out.println("1. Add\n2. List\n3. Exit");
	String choice = sc.nextLine();
	if (choice.equals("3")) { break; }
	try {
	if (choice.equals("1")) {
	System.out.print("Name: "); service.add(sc.nextLine());
	} else if (choice.equals("2")) { System.out.println(service.all()); }
	else { System.out.println("Invalid choice"); }
	} catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
	}
	}
	}

	ItemServiceTest.java

Code	import org.junit.jupiter.api.Test;
	import static org.junit.jupiter.api.Assertions.*;
	public class ItemServiceTest {
	@Test void addIncreasesCount() {
	ItemService service = new ItemService();
	service.add("Book"); assertEquals(1,service.count());
	}
	@Test void blankNameThrows() {
	assertThrows(IllegalArgumentException.class, () -> new ItemService().add(" "));
	}
	@Test void listContainsAddedItem() {
	ItemService service = new ItemService(); service.add("Pen");
	assertEquals("Pen",service.all().get(0).name());
	}
	}

	Place JUnit Platform Console Standalone 1.10.2 in lib/junit.jar. javac -d out Item.java ItemService.java ItemApp.java javac -cp "out;lib/junit.jar" -d out ItemServiceTest.java

	java -jar lib/junit.jar execute --class-path out --scan-class-path On Linux/macOS replace the classpath semicolon with a colon.
Output	

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB14\Q1"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>javac -d out Item.java ItemService.java ItemApp.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>javac -cp "out;lib\junit.jar" -d out ItemServiceTest.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>java -jar lib\junit.jar execute --class-path out --scan-class-path
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

.
+-- JUnit Jupiter [OK]
|  +-- ItemServiceTest [OK]
|     +-- addIncreasesCount() [OK]
|     +-- blankNameThrows() [OK]
|     +-- listContainsAddedItem() [OK]
+-- JUnit Vintage [OK]
+-- JUnit Platform Suite [OK]

Test run finished after 72 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         3 tests found           ]
[         0 tests skipped         ]
[         3 tests started         ]
[         0 tests aborted         ]
[         3 tests successful      ]
[         0 tests failed          ]

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>
```

2)	(For Advanced Learners – home) Organise that app into packages, build a runnable JAR, and add a test that saves and reloads its data unchanged.
	model/Item.java
Code	package model; import java.io.Serializable; public record Item(String name) implements Serializable {}
	service/ItemService.java

Code	package service;
	import model.Item;
	import java.io.*;
	import java.nio.file.*;
	import java.util.ArrayList;
	public class ItemService {
	  private ArrayList<Item> items = new ArrayList<>();
	  public void add(String name) {
	  if (name == null || name.isBlank()) { throw new IllegalArgumentException("Name required"); }
	  items.add(new Item(name));
	  }
	  public void save(Path file) throws IOException {
	  try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(file))) {
	  out.writeObject(items);
	  }
	  }
	  @SuppressWarnings("unchecked")
	  public void load(Path file) throws IOException, ClassNotFoundException {
	  try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(file))) {
	  items = (ArrayList<Item>) in.readObject();
	  }
	  }
	  public int count() { return items.size(); }
	  public ArrayList<Item> all() { return new ArrayList<>(items); }
	}

	app/ItemApp.java

Code	package app;
	import service.ItemService;
	import java.util.Scanner;
	public class ItemApp {
	public static void main(String[] args) {
	ItemService service = new ItemService();
	Scanner sc = new Scanner(System.in);
	while (true) {
	System.out.println("1. Add\n2. List\n3. Exit");
	String choice = sc.nextLine();
	if (choice.equals("3")) { break; }
	try {
	if (choice.equals("1")) {
	System.out.print("Name: "); service.add(sc.nextLine());
	} else if (choice.equals("2")) { System.out.println(service.all()); }
	else { System.out.println("Invalid choice"); }
	} catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
	}
	}
	}

	test/ItemServiceTest.java

Code	package test;
	import service.ItemService;
	import java.nio.file.*;
	import org.junit.jupiter.api.Test;
	import static org.junit.jupiter.api.Assertions.*;
	public class ItemServiceTest {
	@Test void addIncreasesCount() {
	ItemService service = new ItemService();
	service.add("Book"); assertEquals(1,service.count());
	}
	@Test void blankNameThrows() {
	assertThrows(IllegalArgumentException.class, () -> new ItemService().add(" "));
	}
	@Test void listContainsAddedItem() {
	ItemService service = new ItemService(); service.add("Pen");
	assertEquals("Pen",service.all().get(0).name());
	}
	@Test void saveAndLoad() throws Exception {
	Path file = Files.createTempFile("items", ".dat");
	try {
	ItemService first = new ItemService(); first.add("Pen"); first.save(file);
	ItemService second = new ItemService(); second.load(file);
	assertEquals(first.all(),second.all());
	} finally { Files.deleteIfExists(file); }
	}
	}

	Build from the folder containing model, service, app and test. javac -d out model/Item.java service/ItemService.java app/ItemApp.java jar --create --file items.jar --main-class app.ItemApp -C out . java -jar items.jar javac -cp "out;lib/junit.jar" -d test-out test/ItemServiceTest.java java -jar lib/junit.jar execute --class-path "out;test-out" --scan-class-path On Linux/macOS use a colon in classpaths.

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB14\Q2"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>javac -d out model\Item.java service\ItemService.java app\ItemApp.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>jar --create --file items.jar --main-class app.ItemApp -C out .

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>javac -cp "out;lib\junit.jar" -d test-out test\ItemServiceTest.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>java -jar lib\junit.jar execute --class-path "out;test-out" --scan-class-path
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

.
+-- JUnit Jupiter [OK]
|  +-- ItemServiceTest [OK]
|     +-- addIncreasesCount() [OK]
|     +-- blankNameThrows() [OK]
|     +-- saveAndLoad() [OK]
|     +-- listContainsAddedItem() [OK]
+-- JUnit Vintage [OK]
+-- JUnit Platform Suite [OK]

Test run finished after 183 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         4 tests found           ]
[         0 tests skipped         ]
[         4 tests started         ]
[         0 tests aborted         ]
[         4 tests successful      ]
[         0 tests failed          ]

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>
```